

Perceptron

Yongmei Tan

ymtan@bupt.edu.cn

感知机

- 感知机模型
- 感知机学习策略
- 感知机学习算法

感知机模型

- 二类分类的线性分类模型，其输入为实例的特征向量，输出为实例的类别，取+1和-1二值。
- 对应于输入空间（特征空间）中将实例划分为正负两类的分离超平面，属于判别模型。
- 旨在求出将训练数据进行线性划分的分离超平面，为此导入基于误分类的损失函数，利用梯度下降法对损失函数进行极小化，求得感知机模型。

感知机模型

- 感知机的定义

输入空间（特征空间）是 $\mathcal{X} \subseteq \mathbf{R}^n$ ，输出空间是 $\mathcal{Y} = \{+1, -1\}$ ，输入 $\mathbf{x} \in \mathcal{X}$ 表示实例的特征向量，对应输入空间（特征空间）的点；输出 $y \in \mathcal{Y}$ 表示实例的类别。

由输入空间到输出空间的函数：

$$f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

称为感知机。

其中 $\mathbf{w}$ 和 b 为感知机模型模型参数， $\mathbf{w} \in \mathbf{R}^n$ 叫做权值（weight）或权值向量（weight vector）， $b \in \mathbf{R}$ 叫做偏置（bias）， $\mathbf{w} \cdot \mathbf{x}$ 表示 $\mathbf{w}$ 和 $\mathbf{x}$ 的内积。

感知机模型

- **Sign**符号函数

$$\text{sign}(x) = \begin{cases} +1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

- 感知机是一种线性分类模型，属于判别模型；
- 感知机模型的假设空间是定义在特征空间中的所有线性分类模型（**linear classification model**）或线性分类器（**linear classifier**），即函数集合
$$\{f | f(x) = w \cdot x + b\}$$

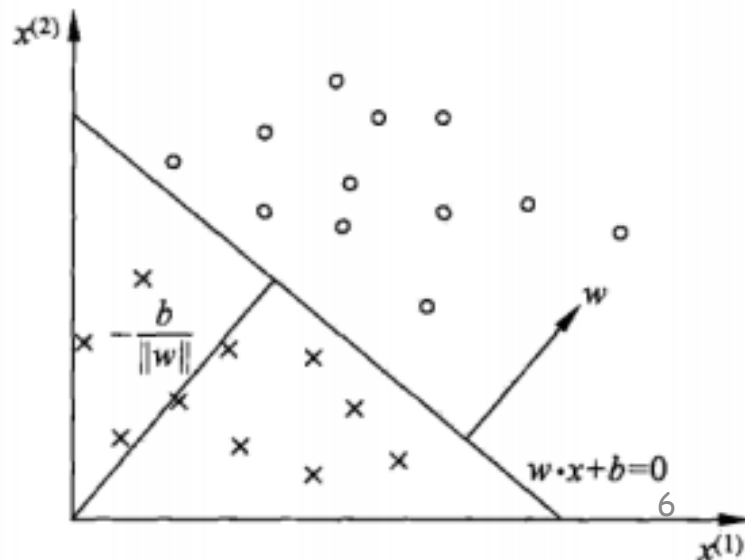
感知机模型

- 感知机有如下几何解释：线性方程

$$w \cdot x + b = 0$$

对应于特征空间 $\mathbf{R}^n$ 中的一个超平面 S ，其中 w 是超平面的法向量， b 是超平面的截距。

- 超平面将特征空间划分为两个部分，位于两部分的点（特征向量）分别被分为正、负两类。因此该平面 S 成为分离超平面（**separating hyperplane**）。



感知机学习策略

- 数据集的线性可分性

- 给定数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

其中 $x_i \in \mathcal{X} = \mathbf{R}^n$, $y_i \in Y = \{+1, -1\}$, $i = 1, 2, \dots, N$,
如果存在某个超平面 S

$$w \cdot x + b = 0$$

- 能够将数据集的正实例点和负实例点完全正确的划分到超平面的两侧，则称数据集 T 为线性可分数据集 (linearly separable data set) ;

- 否则，称数据集 T 线性不可分。

感知机学习策略

- 感知机学习策略

- 损失函数为误分类点的总数，不易优化。
- 损失函数为误分类点到超平面 S 的总距离。
- 输入空间 $\mathbf{R}^n$ 中任一点 x_0 到超平面 S 的距离：

$$\frac{1}{\|w\|} |w \cdot x_0 + b|$$

$\|w\|$ 是 w 的 L_2 范数。

感知机学习策略

- 对于误分类的数据 (x_i, y_i) 来说,
$$-y_i(w \cdot x_i + b) > 0$$

成立。

- 误分类点 x_i 到超平面 S 的距离是
$$-\frac{1}{\|w\|}y_i(w \cdot x_i + b)$$

- 假设超平面 S 的误分类点集合为 M ，那么所有误分类点到超平面 S 的总距离为

$$-\frac{1}{\|w\|} \sum_{x_i \in M} y_i(w \cdot x_i + b)$$

- 不考虑 $\frac{1}{\|w\|}$ ，得到感知机学习的损失函数。

感知机学习策略

- 给定训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

其中, $x_i \in \mathcal{X} = \mathbf{R}^n$, $y_i \in \mathcal{Y} = \{+1, -1\}$, $i = 1, 2, \dots, N$. 感知机 $\text{sign}(w \cdot x + b)$ 学习的损失函数定义为

$$L(w, b) = - \sum_{x_i \in M} y_i (w \cdot x_i + b)$$

其中 M 为误分类点的集合。

$L(w, b)$ 为感知机学习的经验风险函数。

感知机学习算法

- 给定训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

$$x_i \in \mathcal{X} = \mathbf{R}^n, \quad y_i \in \mathcal{Y} = \{-1, 1\}, \quad i = 1, 2, \dots, N,$$

求参数 $\mathbf{w}$, b , 使其为损失函数极小化问题的解

$$\min_{w, b} L(w, b) = - \sum_{x_i \in M} y_i (w \cdot x_i + b)$$

M 为误分类点的集合

感知机学习算法

- 假设误分类点集合 M 是固定的，那么损失函数 $L(w, b)$ 的梯度为

$$\nabla_w L(w, b) = - \sum_{x_i \in M} y_i x_i$$

$$\nabla_b L(w, b) = - \sum_{x_i \in M} y_i$$

- 随机选取一个误分类点 (x_i, y_i) ，对 w, b 进行更新：

$$w \leftarrow w + \eta y_i x_i$$

$$b \leftarrow b + \eta y_i$$

$\eta (0 < \eta \leq 1)$ 是步长，又称学习率 (learning rate)，通过迭代可以将损失函数 $L(w, b)$ 不断减小，直到为0。

感知机学习算法

输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$,
其中 $x_i \in \mathcal{X} = \mathbf{R}^n$, $y_i \in \mathcal{Y} = \{-1, +1\}$, $i = 1, 2, \dots, N$;
学习率 $\eta (0 < \eta \leq 1)$;

输出： w, b ；感知机模型 $f(x) = \text{sign}(w \cdot x + b)$.

(1) 选取初值 w_0, b_0

(2) 在训练集中选取数据 (x_i, y_i)

(3) 如果 $y_i(w \cdot x_i + b) \leq 0$

$$w \leftarrow w + \eta y_i x_i$$

$$b \leftarrow b + \eta y_i$$

(4) 转至 (2)，直至训练集中没有误分类点

感知机学习算法

- 例2.1 正实例点 $x_1=(3,3)^T, x_2=(4,3)^T$, 负实例点 $x_3=(1,1)^T$, 试用感知机学习算法的原始形式求感知机模型 $f(x)=\text{sign}(w \cdot x + b)$, $w=(w^{(1)}, w^{(2)})^T$, $x=(x^{(1)}, x^{(2)})^T$.

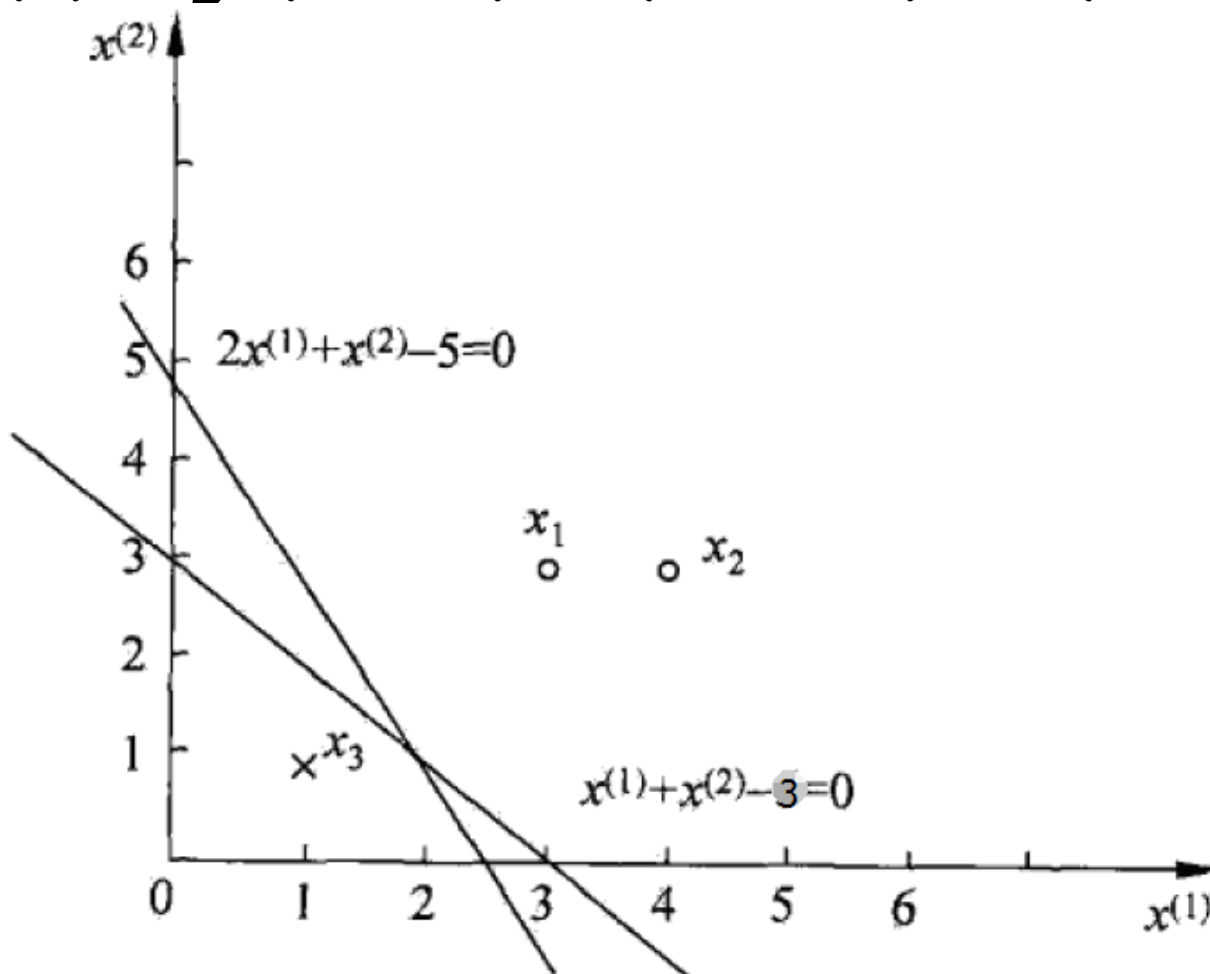


表 2.1 例 2.1 求解的迭代过程

迭代次数	误分类点	w	b	$w \cdot x + b$
0		0	0	0
1	x_1	$(3,3)^T$	1	$3x^{(1)} + 3x^{(2)} + 1$
2	x_3	$(2,2)^T$	0	$2x^{(1)} + 2x^{(2)}$
3	x_3	$(1,1)^T$	-1	$x^{(1)} + x^{(2)} - 1$
4	x_3	$(0,0)^T$	-2	-2
5	x_1	$(3,3)^T$	-1	$3x^{(1)} + 3x^{(2)} - 1$
6	x_3	$(2,2)^T$	-2	$2x^{(1)} + 2x^{(2)} - 2$
7	x_3	$(1,1)^T$	-3	$x^{(1)} + x^{(2)} - 3$
8	0	$(1,1)^T$	-3	$x^{(1)} + x^{(2)} - 3$

感知机学习算法

- 对偶形式

- 将 w 和 b 表示为实例 x_i 和标记 y_i 的线性组合的形式，通过求解其系数而求得 w 和 b ，在原始形式的学习算法中可假设初始值 w_0, b_0 均为0，对误分类点 (x_i, y_i) 通过

$$w \leftarrow w + \eta y_i x_i$$

$$b \leftarrow b + \eta y_i$$

逐步修改 w, b ，设修改 n 次，则 w, b 关于 (x_i, y_i) 的增量分别是 $\alpha_i y_i x_i$ 和 $\alpha_i y_i$ ，这里 $\alpha_i = n_i \eta$ 。

感知机学习算法

最后学习到的 w, b 可以分别为

$$w = \sum_{i=1}^N \alpha_i y_i x_i$$

$$b = \sum_{i=1}^N \alpha_i y_i$$
$$\alpha_i = n_i \eta$$

$\alpha_i \geq 0, \quad i = 1, 2, \dots, N$, 当 $\eta = 1$ 时, n_i 表示第 i 个实例点由于误分而进行更新的次数。

实例点更新次数越多, 意味着它距离分离超平面越近, 也就越难正确的分类。

感知机学习算法

输入：线性可分的数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$,

其中 $x_i \in \mathcal{X} = \mathbf{R}^n$, $y_i \in \mathcal{Y} = \{-1, +1\}$, $i = 1, 2, \dots, N$;

学习率 $\eta (0 < \eta \leq 1)$

输出：感知机模型 $f(x) = \text{sign}(\sum_{j=1}^N \alpha_j y_j x_j \cdot x + b)$.

其中 $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_N)^T$,

(1) $\alpha \leftarrow 0, b \leftarrow 0$

(2) 在训练集中选取数据 (x_i, y_i)

如果 $y_i (\sum_{j=1}^N \alpha_j y_j x_j \cdot x_i + b) \leq 0$

$$\alpha_i \leftarrow \alpha_i + \eta$$

$$b \leftarrow b + \eta y_i$$

(4) 转至 (2) 直到没有误分类数据。

感知机学习算法

- Gram矩阵 (Gram matrix)

$$G = [x_i \cdot x_j]_{N \times N}$$

训练集中的实例间的内积

感知机学习算法

- 例2.2 正实例点 $x_1=(3,3)^T, x_2=(4,3)^T$, 负实例点 $x_3=(1,1)^T$, 试用感知机学习算法对偶形式求感知机模型 $f(x)=\text{sign}(w \cdot x + b), w=(w^{(1)}, w^{(2)})^T, x=(x^{(1)}, x^{(2)})^T$.

(1) 取 $\alpha_i=0, i=1,2,3, b=0, \eta=1$

(2) 计算 Gram 矩阵

$$G = \begin{bmatrix} 18 & 21 & 6 \\ 21 & 25 & 7 \\ 6 & 7 & 2 \end{bmatrix}$$

(3) 误分条件

$$y_i \left(\sum_{j=1}^N \alpha_j y_j x_j \cdot x_i + b \right) \leq 0$$

参数更新

$$\alpha_i \leftarrow \alpha_i + 1, b \leftarrow b + y_i$$

(4) 迭代. 过程从略, 结果列于表 2.2.

表 2.2 例 2.2 求解的迭代过程

k	0	1	2	3	4	5	6	7
α_1	0	1	2	3	4	5	6	7
α_2	0	1	2	3	4	5	6	7
α_3	0	1	2	3	4	5	6	7
b	0	1	2	3	4	5	6	7

感知机学习算法

- 例2.2 正实例点 $x_1=(3,3)^T, x_2=(4,3)^T$, 负实例点 $x_3=(1,1)^T$, 试用感知机学习算法对偶形式求感知机模型 $f(x)=\text{sign}(w \cdot x + b), w=(w^{(1)}, w^{(2)})^T, x=(x^{(1)}, x^{(2)})^T$.

(1) 取 $\alpha_i=0, i=1,2,3, b=0, \eta=1$

(2) 计算 Gram 矩阵

$$G = \begin{bmatrix} 18 & 21 & 6 \\ 21 & 25 & 7 \\ 6 & 7 & 2 \end{bmatrix}$$

(3) 误分条件

$$y_i \left(\sum_{j=1}^N \alpha_j y_j x_j \cdot x_i + b \right) \leq 0$$

参数更新

$$\alpha_i \leftarrow \alpha_i + 1, b \leftarrow b + y_i$$

表 2.2 例 2.2 求解的迭代过程

k	0	1	2	3	4	5	6	7
		x_1	x_3	x_3	x_3	x_1	x_3	x_3
α_1	0	1	1	1	1	2	2	2
α_2	0	0	0	0	0	0	0	0
α_3	0	0	1	2	3	3	4	5
b	0	1	0	-1	-2	-1	-2	-3

(4) 迭代. 过程从略, 结果列于表 2.2.

(5)

$$w = 2x_1 + 0x_2 - 5x_3 = (1, 1)^T$$

$$b = -3$$

分离超平面

$$x^{(1)} + x^{(2)} - 3 = 0$$

表 2.2 例 2.2 求解的迭代过程

k	0	1	2	3	4	5	6	7
		x_1	x_3	x_3	x_3	x_1	x_3	x_3
α_1	0	1	1	1	1	2	2	2
α_2	0	0	0	0	0	0	0	0
α_3	0	0	1	2	3	3	4	5
b	0	1	0	-1	-2	-1	-2	-3

homework

训练数据集中，两类训练样本如下，用感知机算法求感知机模型 $f(x)=\text{sign}(w \bullet x+b)$ ，学习率 $\eta=1$ 。

正实例点： $x_1=(0, 0, 0)^T$, $x_2=(1, 0, 0)^T$, $x_3=(1, 0, 1)^T$, $x_4=(1, 1, 0)^T$

负实例点： $x_5=(0, 0, 1)^T$, $x_6=(0, 1, 1)^T$, $x_7=(0, 1, 0)^T$, $x_8=(1, 1, 1)^T$

Thank You!

范数

常用范数

这里以 $\mathbb{C}^n$ 空间为例， $\mathbb{R}^n$ 空间类似。

最常用的范数就是p-范数。若 $x = [x_1, x_2, \dots, x_n]^T$ ，那么

$$\|x\|_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{\frac{1}{p}}$$

可以验证p-范数确实满足范数的定义。其中三角不等式的证明不是平凡的，这个结论通常称为[闵可夫斯基\(Minkowski\)](#)不等式。

当p取1, 2, ∞ 的时候分别是以下几种最简单的情形：

$$1\text{-范数: } \|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$$

$$2\text{-范数: } \|x\|_2 = (|x_1|^2 + |x_2|^2 + \dots + |x_n|^2)^{1/2}$$

$$\infty\text{-范数: } \|x\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|)$$

其中2-范数就是通常意义下的距离。

点到平面距离公式

给定平面 $Ax + By + Cz + D = 0$

求平面外一点 $Q(x_0, y_0, z_0)$ 到平面的距离 d

解：任取平面内一点 $P(x, y, z)$ 连接 PQ ,

过 P 做平面的法向量 $\vec{n} = (A, B, C)$

可知 Q 到平面的距离 d 恰是 PQ 在法向量 $\vec{n}$ 上的投影长度。

$$d = |\overrightarrow{PQ}| \cdot \cos \theta$$

$$= \frac{|\vec{n}|}{|\vec{n}|} \cdot |\overrightarrow{PQ}| \cdot \cos \theta \quad (\text{乘以 } |\vec{n}| \text{ 再除以 } |\vec{n}|)$$

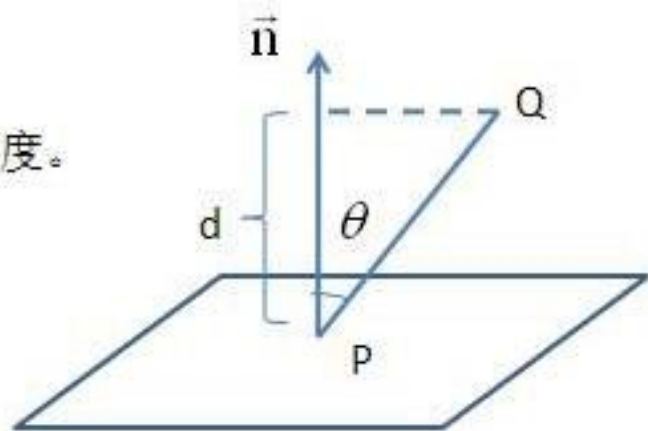
$$= \frac{\overrightarrow{PQ} \cdot \vec{n}}{|\vec{n}|} \quad (\text{等式上边正好是 } \overrightarrow{PQ} \text{ 和 } \vec{n} \text{ 的点积})$$

$$= \frac{A(x_0 - x) + B(y_0 - y) + C(z_0 - z)}{\sqrt{A^2 + B^2 + C^2}} \quad (\overrightarrow{PQ} = (x_0 - x, y_0 - y, z_0 - z))$$

$$= \frac{Ax_0 + By_0 + Cz_0 - (Ax + By + Cz)}{\sqrt{A^2 + B^2 + C^2}} \quad (\text{展开})$$

$$= \frac{Ax_0 + By_0 + Cz_0 + D}{\sqrt{A^2 + B^2 + C^2}} \quad (P \text{ 在平面内所以 } Ax + By + Cz = -D)$$

$$= \frac{|Ax_0 + By_0 + Cz_0 + D|}{\sqrt{A^2 + B^2 + C^2}} \quad (\text{为保证结果为正, 在等式上加绝对值})$$



根据定义,对x轴偏导数 $\frac{df(x,y)}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x+\Delta x, y) - f(x, y)}{\Delta x}$

对y轴偏导数 $\frac{df(x,y)}{dy} = \lim_{\Delta y \rightarrow 0} \frac{f(x, y+\Delta y) - f(x, y)}{\Delta y}$

方向导数(设l与x轴夹角为 θ) = $\frac{df(x,y)}{dl}$

$$= \lim_{t \rightarrow 0^+} \frac{f(x+t \cos \theta, y+t \sin \theta) - f(x, y)}{t}$$

$$= \lim_{t \rightarrow 0^+} \frac{f(x+t \cos \theta, y+t \sin \theta) - f(x, y+t \sin \theta) + f(x, y+t \sin \theta) - f(x, y)}{t}$$

$$= \lim_{t \rightarrow 0^+} \frac{f(x+t \cos \theta, y+t \sin \theta) - f(x, y+t \sin \theta)}{t} + \lim_{t \rightarrow 0^+} \frac{f(x, y+t \sin \theta) - f(x, y)}{t}$$

$$= \cos \theta \lim_{t \cos \theta \rightarrow 0^+} \frac{f(x+t \cos \theta, y+t \sin \theta) - f(x, y+t \sin \theta)}{t \cos \theta} + \sin \theta \lim_{t \sin \theta \rightarrow 0^+} \frac{f(x, y+t \sin \theta) - f(x, y)}{t \sin \theta}$$

$$= \frac{\partial f}{\partial x} \cos \theta + \frac{\partial f}{\partial y} \sin \theta$$

梯度(记作 $\text{grad}f(x, y)$ 或 $\nabla f(x, y)$) = $\frac{\partial f}{\partial x} i + \frac{\partial f}{\partial y} j$,

(i, j 分别是x轴和y轴的单位向量)

即函数的偏导数是函数对坐标轴的导数,

反应的是函数沿坐标轴方向的变化率,

方向导数是函数沿着某一个与坐标轴成夹角的方向的导数,

反应的是函数沿着某个方向的变化率,已知夹角和偏导数

可以算出方向导数

梯度是与偏导数有关的一个向量,算出偏导数即可。

梯度

- 在向量微积分中，标量场的梯度是一个向量场。标量场中某一点上的梯度指向标量场增长最快的方向，梯度的长度是这个最大的变化率。
- 在单变量的实值函数的情况，梯度只是导数，或者，对于一个线性函数，也就是线的斜率。
- 梯度一词有时用于斜度，也就是一个曲面沿着给定方向的倾斜程度。可以通过取向量梯度和所研究的方向的点积来得到斜度。梯度的数值有时也被称为梯度。
- 二元函数的情形，设函数 $z=f(x,y)$ 在平面区域 D 内具有一阶连续偏导数，则对于每一点 $P(x,y) \in D$ ，都可以定出一个向量 $(\delta f / \delta x) * i + (\delta f / \delta y) * j$ ，这向量称为函数 $z=f(x,y)$ 在点 $P(x,y)$ 的梯度，记作 $\text{grad}[f(x,y)]$
- 三元函数 $(\delta f / \delta x) * i + (\delta f / \delta y) * j + (\delta f / \delta z) * k$ 记为 $\text{grad}[f(x,y,z)]$

一个标量函数的梯度记为：

$\nabla\varphi$ 或者 $grad\varphi$

∇ 表示微分算子。

在三维直角坐标系中表示为：

$$\nabla\varphi = \left(\frac{\partial\varphi}{\partial x}, \frac{\partial\varphi}{\partial y}, \frac{\partial\varphi}{\partial z} \right)$$

梯度下降

- 是一个最优化算法，也称为最速下降法。
- 求解无约束优化问题最简单和最古老的方法之一，虽然现在已经不具有实用性，但是许多有效算法都是以它为基础进行改进和修正而得到的。
- 用负梯度方向为搜索方向，最速下降法越接近目标值，步长越小，前进越慢。

举一个非常简单的例子，如求函数 $f(x) = x^2$ 的最小值。

利用梯度下降的方法解题步骤如下：

1、求梯度， $\nabla = 2x$

2、向梯度相反的方向移动 x ，如下

$x \leftarrow x - \gamma \cdot \nabla$ ，其中， γ 为步长。如果步长足够小，则可以保证每一次迭代都在减小，但可能导致收敛太慢，如果步长太大，则不能保证每一次迭代都减少，也不能保证收敛。

3、循环迭代步骤2，直到 x 的值变化到使得 $f(x)$ 在两次迭代之间的差值足够小，比如0.00000001，也就是说，直到两次迭代计算出来的 $f(x)$ 基本没有变化，则说明此时 $f(x)$ 已经达到局部最小值了。

4、此时，输出 x ，这个 x 就是使得函数 $f(x)$ 最小时的 x 的取值。